



Form API Submission How-To

Create a submission and export it as a rendered PDF

Purpose

Use the public form API to create a saved form submission, then export that saved submission through the form PDF renderer.

This guide is written in markdown that the Tyria markdown PDF exporter can render cleanly. It uses plain headings, paragraphs, lists, quotes, and fenced code blocks.

What You Need

- A workspace API key with `forms.write`.
- `forms.read` if you need to discover form ids, question ids, or verify the created entry.
- The target `formId`.
- The exact question ids from the form template.
- An authenticated app session, signed receipt token, or workflow action for PDF export. The `/api/v1` API-key surface creates and reads entries; the saved-submission PDF route is an app/form export route.

Do not put API keys, PHI, or customer PII into shared screenshots, support tickets, or example payloads. Keep example data synthetic.

Endpoint

Create a form submission with:

```
POST /api/v1/forms/{formId}/entries
```

Use the canonical API host:

```
https://www.tyriacore.app
```

Call the `www` host directly for API requests. The bare `tyriacore.app` host redirects to `www.tyriacore.app`, and some API clients handle redirects on POST differently.

Use:



Form API Submission How-To

Create a submission and export it as a rendered PDF

```
Authorization: Bearer YOUR_API_KEY  
Content-Type: application/json
```

The key must include:

```
forms.write
```

Request Body

The body is a JSON object.

- `answers` is required. It must be an object keyed by form question id.
- `id` is optional. Use it only when the caller needs idempotent external ids.
- `submittedAt` is optional. Use an ISO timestamp. If omitted, Tyria stores the current server time.
- `submittedBy` is optional. It can include `userId`, `email`, and `name`.
- `shareId` is optional. Use it only when the submission is associated with a share or access link.
- `meta` is optional. Use it for source attribution and linked entity context.

Find the Form and Question IDs

If you do not already have the template id, list forms:

```
curl -X GET "https://www.tyriacore.app/api/v1/forms?limit=50" \  
-H "Authorization: Bearer YOUR_API_KEY"
```

The response includes form records with a `questions` array. Copy:

- `data[].id` for the `formId`.
- `data[].questions[].id` for each answer key.
- `data[].questions[].options` or `statusOptions` for constrained answer values.

Create the Submission

This example creates an entry for a standard form and links the saved response to a client timeline.



Form API Submission How-To

Create a submission and export it as a rendered PDF

```
curl -X POST "https://www.tyriacore.app/api/v1/forms/form_123/entries" \  
-H "Authorization: Bearer YOUR_API_KEY" \  
-H "Content-Type: application/json" \  
-d '{  
  "submittedAt": "2026-06-26T15:30:00.000Z",  
  "submittedBy": {  
    "email": "ops@example.com",  
    "name": "Operations"  
  },  
  "answers": {  
    "customer_name": "Jordan Lee",  
    "install_date": "2026-07-15",  
    "service_level": "Premium",  
    "approval_status": "Approved"  
  },  
  "meta": {  
    "source": "public-api",  
    "externalSubmissionId": "crm-form-00042",  
    "linkedEntity": {  
      "entityType": "client",  
      "entityId": "client_123",  
      "entityLabel": "Jordan Lee"  
    }  
  }  
}'
```

Successful responses return 201:



Form API Submission How-To

Create a submission and export it as a rendered PDF

```
{
  "data": {
    "id": "entry_123",
    "formId": "form_123",
    "submittedAt": "2026-06-26T15:30:00.000Z",
    "submittedBy": {
      "email": "ops@example.com",
      "name": "Operations"
    },
    "answers": {
      "customer_name": "Jordan Lee",
      "install_date": "2026-07-15",
      "service_level": "Premium",
      "approval_status": "Approved"
    },
    "meta": {
      "source": "public-api",
      "externalSubmissionId": "crm-form-00042",
      "linkedEntity": {
        "entityType": "client",
        "entityId": "client_123",
        "entityLabel": "Jordan Lee"
      }
    }
  }
}
```

Save `data.id`. That id is the submission id used by response lookup and PDF export paths.

JavaScript Example



Form API Submission How-To

Create a submission and export it as a rendered PDF

```
const baseUrl = 'https://www.tyriacore.app';
const formId = 'form_123';
const apiKey = process.env.TYRIA_API_KEY;

const response = await fetch(`${baseUrl}/api/v1/forms/${formId}/entries`, {
  method: 'POST',
  headers: {
    Authorization: `Bearer ${apiKey}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    submittedBy: {
      email: 'ops@example.com',
      name: 'Operations',
    },
    answers: {
      customer_name: 'Jordan Lee',
      install_date: '2026-07-15',
      service_level: 'Premium',
      approval_status: 'Approved',
    },
    meta: {
      source: 'public-api',
      externalSubmissionId: 'crm-form-00042',
      linkedEntity: {
        entityType: 'client',
        entityId: 'client_123',
        entityLabel: 'Jordan Lee',
      },
    },
  }),
});

if (!response.ok) {
  throw new Error(`Submission failed: ${response.status} ${await response.text()}`);
}

const created = await response.json();
console.log(created.data.id);
```

Link the Submission to a Timeline

To attach the saved response to a profile, deal, or event timeline, include linked entity metadata.

Use the nested shape:



Form API Submission How-To

Create a submission and export it as a rendered PDF

```
{
  "meta": {
    "linkedEntity": {
      "entityType": "client",
      "entityId": "client_123",
      "entityLabel": "Jordan Lee"
    }
  }
}
```

Supported `entityType` values are:

- `lead`
- `prospect`
- `client`
- `deal`
- `sales-event`

Aliases such as `sales_event` and `salesEvent` are normalized to `sales-event`.

Validation Rules

The API validates the submitted answers against the saved form question definitions.

- The `answers` value must be an object.
- Answer keys should match question ids from the template.
- Select, multi-select, and status answers must use configured values.
- Date and datetime answers must be parseable date values.
- Number answers must be valid numbers.
- Email and URL answers must match their expected formats.
- Public API form templates do not support signature or attachment questions.

Validation errors return `400` with an error envelope. Fix the field path shown in the response, then retry with corrected data.

Verify the Created Entry

Fetch the entry by id:



Form API Submission How-To

Create a submission and export it as a rendered PDF

```
curl -X GET "https://www.tyriacore.app/api/v1/forms/form_123/entries/entry_123" \  
-H "Authorization: Bearer YOUR_API_KEY"
```

List recent entries:

```
curl -X GET "https://www.tyriacore.app/api/v1/forms/form_123/entries?limit=20" \  
-H "Authorization: Bearer YOUR_API_KEY"
```

Export the Saved Submission as PDF

After the entry is created, export the same saved response through the form PDF renderer.

For an authenticated app session, the inline PDF route is:

```
GET /api/forms/{formId}/responses/{submissionId}/pdf
```

For the example above:

```
GET /api/forms/form_123/responses/entry_123/pdf
```

That route renders the stored response with the shared form export helper. When the PDF service is configured, it uses the styled HTML renderer; otherwise it falls back to the local PDF generator.

For automated email delivery, use the workflow action:

Export & Send Submission PDF

Provide:

- `formId`
- `submissionId`
- recipient email address or addresses



Form API Submission How-To

Create a submission and export it as a rendered PDF

Markdown in the Exported PDF

Markdown rendered inside the form submission PDF comes from `content-text` questions on the form template. The submission supplies answer values that those content blocks can reference with variables.

Example template question:

```
{
  "id": "summary_copy",
  "type": "content-text",
  "label": "Submission Summary",
  "content": "## Submission Summary

Customer: **{{customer_name}}**

- Requested install date: {{install_date}}
- Service level: {{service_level}}"
}
```

Example submitted answers:

```
{
  "answers": {
    "customer_name": "Jordan Lee",
    "install_date": "2026-07-15",
    "service_level": "Premium"
  }
}
```

The renderer resolves `{{customer_name}}`, `{{install_date}}`, and `{{service_level}}` from the saved answers before generating the PDF.

Renderer-Safe Markdown

Use this subset for content intended to render cleanly in the form PDF and standalone markdown PDF exporter:



Form API Submission How-To

Create a submission and export it as a rendered PDF

- Headings with `#`, `##`, and `###`.
- Paragraphs separated by blank lines.
- Unordered lists with `- item`.
- Ordered lists with `1. item`.
- Blockquotes with `>`.
- Fenced code blocks with triple backticks.
- Inline links, inline code, bold, and italic text.

Avoid:

- Raw HTML.
- Markdown tables.
- Deeply nested lists.
- Very wide code blocks.
- Images that require an unauthenticated browser fetch.

Troubleshooting

If the create request returns `401`, check that the API key is active and sent as a Bearer token.

If the response body is the app sign-in page HTML instead of JSON, check that the request is going to `https://www.tyriacore.app` and includes the `Authorization: Bearer YOUR_API_KEY` header.

If the create request returns `403`, check that the API key includes `forms.write`.

If the create request returns `404`, check that the form exists in the workspace and is a standard public API supported form.

If the create request returns `400 Invalid field value`, compare each answer value to the matching question definition.

If the PDF route returns an auth error, use an authenticated app session, a valid signed PDF token, or the workflow PDF email action instead of the `/api/v1` API key.

If markdown does not appear formatted in the PDF, confirm the markdown is in a `content-text` question's `content` field. Normal answer text is printed as answer text, not parsed as markdown.